Optimising Attention on a CPU

CS6886 Systems for Deep Learning — Assignment 5

AMD Ryzen 7 6800H (8C/16T Zen 3+), 16 GB DDR5-4800, Windows 11. gcc 14.1.0, `-O3 -march=native`. All numbers measured on this machine.

At N=4096, d=64 the naive kernel runs at 1.70 GFLOP/s. The final single-threaded kernel runs at 51.9 GFLOP/s, a 30× speed-up. Sixteen threads reach 251 GFLOP/s, a 148× speed-up. All variants pass the check.

---

1 Machine

I read the specification from SMBIOS and CPUID. I measured the limits with `tools/probe.c`. Table 1 gives the result.

Table 1: Machine specification. "Measured" means a microbenchmark, not a datasheet value.

CPU	AMD Ryzen 7 6800H, Zen 3+ (family 0x19, model 0x44)
Cores / threads	8 physical, 16 logical, 2 CCX × 4 cores. No P/E split.
Clock	3.20 GHz base, 4.75 GHz rated boost. **Measured 2.38 GHz 1-core, 2.02 GHz all-core.**
SIMD	AVX2 + FMA3, 256-bit, 2 FMA units per core, 16 YMM registers. No AVX-512.
Caches	L1D 32 KiB/core, L2 512 KiB/core, L3 16 MiB (8 MiB per CCX), 64 B lines.
Memory	16 GB DDR5-4800 SO-DIMM, 2 channels. DRAM latency 144 ns.

Zen 3+ retires two 256-bit FMAs per cycle per core. One FMA is 8 lanes × 2 FLOP, so $P_{\text{core}} = 32f$:

$$P_1 = 32 \times 2.38 = 76.5 \text{ GFLOP/s}, \qquad P_8 = 32 \times 8 \times 2.02 = 495 \text{ GFLOP/s}$$

$$B_{\text{theor}} = 4800\,\text{MT/s} \times 8\,\text{B} \times 2 = 76.8 \text{ GB/s}, \quad B_1 = 21.4 \text{ GB/s}, \quad B_{16} = 35.2 \text{ GB/s}$$

$$R_1 = 76.5/21.4 = 3.57 \text{ FLOP/B}, \qquad R_8 = 495/35.2 = 14.06 \text{ FLOP/B}$$

The part runs at half its rated boost. The clock settles at 2.38 GHz, not 4.75 GHz. Three checks exclude a measurement error. The OS is on the *Best performance* overlay. The power plan allows 100 % processor state on AC. Throughput and latency-based clock estimates agree within 1 %. The platform power-limits the APU. I therefore report efficiency against the measured 76.5 GFLOP/s. Against the datasheet 152 GFLOP/s every figure would halve.

2 Naive baseline

Table 2: Naive kernel. Median of 5 runs after 1 warm-up.

N/d	time (ms)	GFLOP/s	% measured peak	% rated peak
1024/64	138.43	1.94	2.5	1.3
2048/64	571.13	1.88	2.5	1.2
4096/64	2520.70	1.70	2.2	1.1
4096/128	7760.07	1.11	1.5	0.7

Scaling with N. The work grows as $4N^2d$. Time grows faster. Doubling N costs 4.1×, then 4.4×, against the 4× that FLOPs predict. The score matrix explains the gap. At N=2048 it is 16 MB and fits the 16 MB L3. At N=4096 it is 64 MB, so each pass is a DRAM round trip.

Scaling with d. Doubling d costs 3.1×, not 2×. Step 3 walks j with stride d. At d=64 the stride is 256 B and a line feeds four floats. At d=128 it is 512 B, so every load pulls its own line and 15 of 16 bytes are wasted.

Where the time goes. S V dominates at every size: 61 % at 1024/64, rising to 76 % at 4096/128. gcc will not vectorise it: `-fopt-info-vec` reports *"not suitable for strided load"*. Each element costs a scalar load plus one step of a 4-cycle add chain. $QK^\top$ is vectorised but reaches only 2.4–2.8 GFLOP/s, because one accumulator serialises the reduction. Softmax is 1–4 % here. It becomes the largest term once the GEMMs are fixed.

3 Optimisation

(a) Loop reordering. Transpose K once. Then interchange the loops so the innermost loop is an axpy. An axpy has no loop-carried dependency, so it vectorises without `-ffast-math`:

```
S[i][:] += Q[i][k] * KT[k][:]      k outer, j inner
O[i][:] += S[i][j] * V[j][:]       j outer, k inner
```

Table 3: All variants, one session. ms and speed-up over naive. Every variant PASSes the check (max relative error 1.1×10^{-6} to 5.0×10^{-6}).

	1024/64		2048/64		4096/64		4096/128	
variant	ms	sp.	ms	sp.	ms	sp.	ms	sp.
`loops`	18.82	7.4	83.57	6.8	324.97	7.8	577.04	13.4
`tiled`	11.52	12.0	58.80	9.7	223.18	11.3	288.93	26.9
`fused`	15.62	8.9	57.47	9.9	236.12	10.7	527.09	14.7
`best`	4.95	28.0	19.69	29.0	82.72	30.5	168.38	46.1
`omp (16 thr)`	1.89	73.4	5.38	106.1	17.08	147.6	32.17	241.3
`naive`	138.43	1.0	571.13	1.0	2520.70	1.0	7760.07	1.0

Result: 7.8×. Nothing is blocked. Each row re-streams all of $K^\top$ and V from L3, about 8 GB at N=4096. The kernel is L3-bound, not FP-bound.

(b) Tiling. $QK^\top$ uses a 4×16 micro-tile. Eight YMM accumulators, k innermost. Per k that is 8 FMA (128 FLOP) against two 32 B loads plus four broadcasts, so 3 load cycles cover 4 FMA cycles. S V is blocked over the reduction. A 64-key slab of V is 16 KB and stays in L1 while all output rows reuse it, which keeps the d accumulators in registers. Three measurements decided the details.

- gcc will not keep the register tile in registers. In plain C it vectorises the wrong axis and spills. The tile works only with AVX2 intrinsics.
- A 16 KB row is a multiple of the L1 size, so an unpadded block of rows maps onto one L1 set and thrashes. 64 B of padding per row took this variant from 370.6 to 307.0 ms at 4096/64.
- One store per FMA caps a kernel at half of peak. Keeping the output tile in L1 reached 21 %. Register accumulators reach 62 %.

Result: 11.3×. The kernels split 23/49/27 % ($QK^\top$/softmax/S V). $QK^\top$ runs at 72 % of peak. Two limits now dominate. Scalar `expf` is 49 % of kernel time. A further 57 ms, 26 % of wall time, is page faults on the 67 MB score matrix.

(c) Fusion. Row i of O needs only row i of S. I keep one score row in a 16 KB L1 buffer and run the three steps over it. The $N \times N$ matrix is never built. Result: 10.7×, which is *behind* `attn_tiled` (236 against 223 ms), and far behind at d=128 (527 against 289 ms). Fusion is not a win here. It saves about 200 MB of score traffic. It costs 4 GB of L3 traffic at d=64 (V re-read per row), and 8 GB at d=128. Fusion removes the right bottleneck and adds a bigger one.

(d) Fastest single-threaded. Two changes. First, block 64 queries. The score tile stays in cache, so there is no 67 MB allocation, and V is read once per block, not once per row. Second, replace `expf` with an AVX2 exponential. Split $y = x \log_2 e$ into $n + f$ with $|f| \le 0.5$, evaluate 2^f by a degree-5 polynomial, and build 2^n from the exponent field. Measured error 5.1×10^{-6} over $x \in [-80, 0]$; 8.4× faster than libm (16.8 M calls: 9.4 ms against 78.8 ms). Result: 30.5×, and 51.9 GFLOP/s = 68 % of measured peak. That is 2.7× faster than `tiled`.

4 OpenMP

`attn_omp` parallelises the query blocks. $K^\top$ is built once before the team starts. No barrier is needed inside the region. Blocks write disjoint $64 \times d$ slices of O. Shared operands are read-only. Slices are 16 KB apart, so there is no false sharing. One caution: with `OMP_PROC_BIND=close`, Windows packs both threads of a 2-thread run onto one core (53.9 against 45.3 ms at 4096/64). Table 4 is unpinned.

Table 4: Strong scaling. Speed-up is against this binary at one thread.

N/d	threads	time (ms)	speed-up	eff. (%)	GFLOP/s
	1	82.58	1.00	100	52.0
	2	42.68	1.93	97	100.6
4096/64	4	28.15	2.93	73	152.6
	8	19.32	4.27	53	222.3
	16	17.08	4.83	30	251.4
	1	171.08	1.00	100	50.2
	2	93.21	1.84	92	92.2
4096/128	4	54.75	3.12	78	156.9
	8	39.80	4.30	54	215.8
	16	32.17	5.32	33	267.1

Up to four threads the curve is close to linear. Eight threads give 4.27×, or 53 % efficiency. The gap is the clock. The all-core clock is 2.02 GHz against 2.38 GHz, so the best possible 8-thread result here is $8 \times (2.02/2.38) = 8 * 2.02/2.38 : .1f\times$. Sixteen threads give 4.83×. The kernel is FMA-bound and the two SMT threads of a core

share its two FMA pipes. The whole machine reaches 495 GFLOP/s at 16 threads, against $8 \times 76.5 = 612$ from the single-core peak, so SMT adds about a quarter. Efficiency is defined against thread count, so it falls steeply while time still improves.

5 Roofline

The roofline uses measured constants: 76.5 GFLOP/s at 21.4 GB/s on one core. Intensity is useful FLOPs over the bytes a variant must move. The numerator is $4N^2d$. The softmax is charged to time, not FLOPs. The denominators:

$$(4Nd + 3N^2) \times 4 = 205.5 \text{ MB} \Rightarrow 20.9, \qquad 4Nd \times 4 = 4.19 \text{ MB} \Rightarrow 1024, \qquad (4Nd + Nd + \tfrac{N}{64}Nd) \times 4 \Rightarrow 59.4$$

for (naive, loops, tiled), (fused) and (best, omp) respectively.

Table 5: Variants at N=4096, d=64 against their roofline.

variant	AI (FLOP/B)	ms	GFLOP/s	% of relevant peak
`naive`	20.9	2520.70	1.7	2
`loops`	20.9	324.97	13.2	17
`tiled`	20.9	223.18	19.2	25
`fused`	1024.0	236.12	18.2	24
`best`	59.4	82.72	51.9	68
`omp (16)`	59.4	17.08	251.4	51

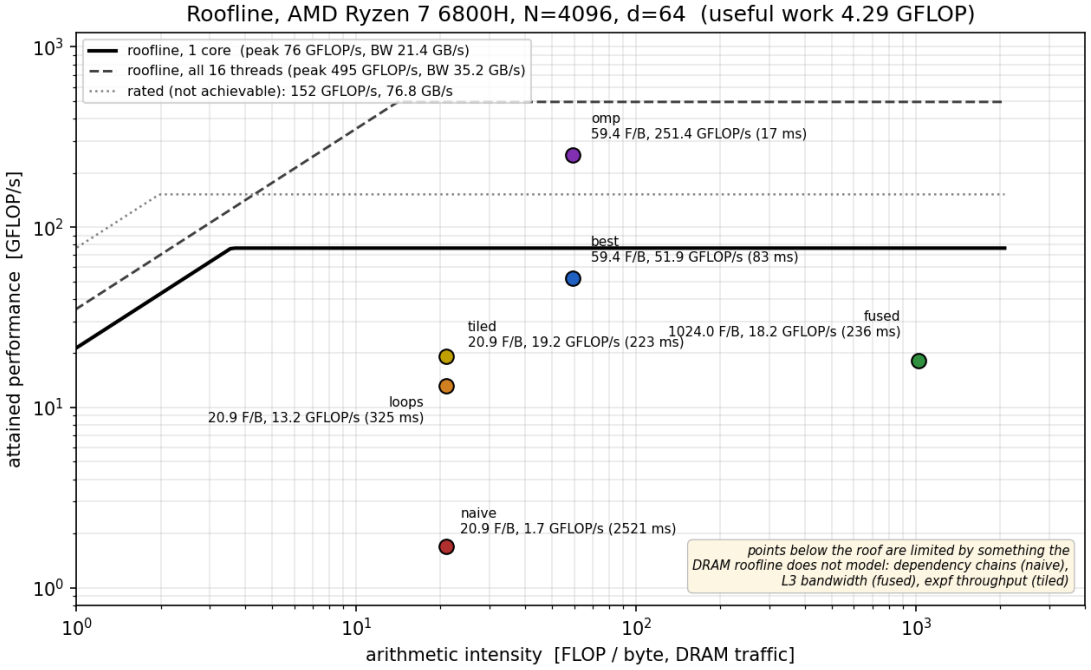


Figure 1: Roofline at N=4096, d=64.

What limits each variant. `naive` sits right of the ridge, where the roofline predicts 76.5 GFLOP/s. It reaches 2.2 %. Neither roof applies: the strided scalar loop costs one dependent add per element. `loops` is compute-bound by the model and 5.8× below it; its real limit is L3 bandwidth. `tiled` has a $QK^\top$ kernel at 72 % of peak, held back by `expf` and page faults. `fused` is 287× past the ridge by the DRAM model, yet reaches only 24 %. Its limit is L3 bandwidth, one level above DRAM and invisible on this plot. `best` is at 68 % of peak; the rest is the softmax, the S V ceiling of 62 %, and compulsory V reads. `omp` lies above the single-core roofline, which is the signature of a kernel using the whole machine. Against the all-core roof it reaches 51 %.

Conformance to the handout

Table 6: Each handout requirement, how it is met, and where the evidence is.

Requirement	How it is met	Evidence
`machine_spec.txt`: all fields, cores incl. P/E, clocks, SIMD, caches, memory size and type	All fields filled from SMBIOS and CPUID, then measured. P/E recorded as “none, homogeneous”.	Table 1

Requirement	How it is met	Evidence
Q1: peak FP32 for one and all cores, memory bandwidth, ridge point, with calculations	Three equation blocks; measured against rated stated.	§1
Q2: time, GFLOP/s, % of peak per input; effect of N and d; where the time goes	Table 2, both peak columns, two scaling paragraphs, phase split and the stride argument.	§2
Q3: same measurements plus speed-up; `make check` passes; what limits each variant	Table 3; all six PASS; one limit statement per variant.	§3
Q3(a–d): reorder loops, cache and register tiling, fuse so $N \times N$ is never stored, maximum single-threaded	Axpy interchange; 4×16 YMM tile; one score row in L1 with no N^2 allocation; query blocking plus AVX2 exp.	§3(a)–(d)
"No BLAS or other math libraries"	Only `libm`, `immintrin.h` and `omp.h`. An include audit confirms it.	final note
Q4: OpenMP, several thread counts, time, speed-up, efficiency, scaling explained	Table 4; the clock and SMT argument.	§4
Q5: roofline from Q1 numbers, every variant at N=4096, d=64, intensity computation, limiting factor	Table 5, Figure 1, byte counts shown, one limit per variant.	§5
Submission structure; `results.csv` rows for each thread count	Structure matches exactly. `results.csv` holds 40 rows.	repository

Notes on reproduction

The handout `Makefile` is submitted unmodified. This box has no `make` (MSYS2 `make` cannot start its signal pipe here) and no `clang`. All numbers therefore come from `tools/bench.py`, which repeats the Makefile steps with gcc 14.1.0 and runs the same check rule. `-march=native` resolves to `znver3`, the ISA the Makefile would select with clang. Each variant reports the median of 5 runs after one warm-up. One table comes from one session, because the part drifts up to 25 % between sessions. Within a session the spread is about ±5 %. Only `libm`, `immintrin.h` and `omp.h` are used: no BLAS, no other math library. Driver code is unchanged from `attn_naive.c` in all five variants. `results.csv` holds 40 rows, including every thread count.